



Spotify Redux

Applicazione simil-Spotify — React · Redux Toolkit · React-Bootstrap

Tipo: Web app front-end (SPA)

Stack: React 19, Redux Toolkit, React-Bootstrap, Vite

Dati: API Deezer (via proxy striveschool-api)

Stato: Progetto completo secondo la specifica

Panoramica

Applicazione minimale ispirata a Spotify, sviluppata come progetto didattico. L'interfaccia è divisa in tre macro-aree:

- **Sidebar** (sinistra, nera): logo, navigazione, barra di ricerca, autenticazione (estetica).
- **Main Content** (centro/destra, gradiente blu scuro): barra di navigazione orizzontale e griglia dei risultati/brani.
- **Player** (fisso in basso, grigio scuro): traccia corrente e controlli di riproduzione (puramente estetici, nessun audio reale).

Stack tecnologico

Libreria	Ruolo nel progetto
React 19 (functional components + hooks)	UI
Redux Toolkit	Stato globale: <code>configureStore</code> , <code>createSlice</code> , <code>createAsyncThunk</code>
React Redux	Binding React ↔ Redux (<code>useSelector</code> , <code>useDispatch</code>)
React-Bootstrap + Bootstrap 5	Grid system e componenti (Container, Row, Col, Card, Offcanvas, Form, Button...)
React Icons	Icone (cuore, home, libreria, burger, controlli player)
Vite	Build tool e dev server
API Deezer (via proxy striveschool-api)	Ricerca brani in tempo reale

Struttura del progetto (scaffolding)

```
src/
├── api/
│   └── deezer.js           # Funzione condivisa searchTracks(query)
├── assets/
│   └── spotify-logo.svg    # Logo usato nella Sidebar
├── redux/
│   ├── store.js           # configureStore
│   └── musicSlice.js       # Slice unico: state, reducers, thunk di ricerca
├── components/
│   ├── Sidebar/           (Sidebar.jsx + Sidebar.css)
│   ├── TopBar/            (TopBar.jsx + TopBar.css)
│   ├── MainContent/       (MainContent.jsx + MainContent.css)
│   ├── SongCard/          (SongCard.jsx + SongCard.css)
│   └── Player/             (Player.jsx + Player.css)
├── App.jsx                # Layout a 3 aree (Bootstrap grid)
├── App.css                # Variabili CSS del tema (colori, gradiente, altezza player)
└── main.jsx               # Entry point, <Provider store={store}>
```

Ogni componente vive nella propria cartella con il proprio foglio di stile dedicato. Le variabili di tema condivise (colori, gradiente, altezza del player) sono centralizzate in `App.css` (`:root`) come custom property CSS.

Stato Redux

```
{
  searchQuery: "",           // stringa corrente della barra di ricerca
  searchResults: [],         // risultati dell'ultima ricerca
  currentTrack: null,        // { id, title, artist, image } in riproduzione nel Player
  favorites: [],             // array di ID brano nei preferiti
  isLoading: false,         // true durante la fetch di ricerca
  isError: false,           // true se la fetch fallisce
}
```

Azioni: `setSearchQuery` , `resetSearch` , `setCurrentTrack` , `toggleFavorite` + thunk asincrono `fetchSearchResults` .

Funzionalità per componente

Sidebar

- Logo "Spotify" cliccabile → `resetSearch()` , riporta l'app allo stato iniziale.
- Home / Your Library: link morti.
- Barra di ricerca: input controllato da Redux; submit (Invio o click su GO) lancia `fetchSearchResults` .
- Sign Up / Login: bottoni puramente estetici.
- Cookie Policy / Privacy: link morti.
- ≥768px: sidebar `sticky` , sempre visibile durante lo scroll. <768px: collassa in un **Offcanvas** apribile da un burger.

TopBar

Cinque link in maiuscolo (TRENDING, PODCAST, MOODS AND GENRES, NEW RELEASES, DISCOVER), tutti dead link.

Main Content & Song Card

- Prima di ogni ricerca: tre sezioni di default precaricate (Metal, Pop-Rock, #HipHop).
- Dopo una ricerca: sezione unica con i risultati, titolata con la query.
- Nessun risultato / errore di rete: messaggio dedicato con pulsante **Riprova**.
- Ogni card: copertina (lazy-loaded), titolo troncato, artista, icona cuore.
- Click sulla card → aggiorna `currentTrack` (Player). Click sul cuore (con `stopPropagation`) → toggle nei preferiti.

Player

Fisso in basso, sempre visibile. Sinistra: miniatura + titolo + artista del brano selezionato. Centro: controlli Shuffle / Previous / Play / Next / Repeat, centrati via CSS grid a 3 colonne. Puramente estetico, nessuna riproduzione audio reale.

Responsive / Mobile-first

- Sidebar → Offcanvas sotto i 768px (burger + logo in barra compatta).
- TopBar, griglia card e Player riducono padding/gap/dimensioni sotto 767.98px e 575.98px.
- Sezioni home (sempre 4 tracce): griglia dedicata 2-per-riga tra 576–768px per evitare elementi "orfani"; la griglia dei risultati di ricerca resta invariata.

Limiti noti

Player estetico: nessun audio viene realmente riprodotto.

Preferiti in memoria: vivono solo nello stato Redux, persi al refresh (persistenza localStorage individuata come estensione futura, non implementata).

API gratuita: striveschool-api (Heroku free tier) può avere un "cold start" lento al primo utilizzo dopo inattività — mitigato dal pulsante Riprova.